

TD N° 3 : Architecture des ordinateurs

Exercice 1 :

- 1) Déclarer trois variables : **a**, **b** et **s** dans les cases mémoires d'adresses respectives 20h, 2FFh et 400h.
- 2) Mettez les valeurs 0x57 et 0xA8 respectivement dans **a** et **b**.
- 3) Faire un organigramme décrivant le processus de l'addition des contenus des **a** et **b** et de la mise du résultat dans **s**.
- 4) Ecrire le programme assembleur répondant à l'organigramme.

Exercice 2 :

- 1) Déclarer les variables **a**, **b** et **r** dans les cases mémoires d'adresses respectives 20h, 2FFh et 400h.
- 2) Faire un organigramme décrivant le processus suivant :
 - ✚ Si **a=b** → **r='E'**
 - ✚ Si **a>b** → **r='S'**
 - ✚ Si **a<b** → **r='I'**
- 3) Ecrire le programme assembleur répondant à l'organigramme.

Exercice 3 : Temporisation avec plusieurs boucles

✚ Temporisation avec deux boucles imbriquées

Soit le code suivant (On rappelle que pour une fréquence f_H de 4MHz) :

	<i>movlw</i>	<i>d'100'</i>		
	<i>movwf</i>	<i>cmpt2</i>	<i>; initialiser compteur2</i>	1 cycle
<i>boucle2</i>				
	<i>movlw</i>	<i>d'249'</i>		1 cycle
	<i>movwf</i>	<i>cmpt1</i>	<i>; initialiser compteur1</i>	1 cycle
<i>boucle1</i>				
	<i>nop</i>		<i>; perdre 1 cycle</i>	1 cycle
	<i>decfsz</i>	<i>cmpt1, f</i>	<i>; décrémenter compteur1</i>	1 ou 2 cycles
	<i>goto</i>	<i>boucle1</i>	<i>; si pas 0, boucler</i>	2 cycles
			<i>; si 0, décrémenter</i>	1 ou 2
	<i>decfsz</i>	<i>cmpt2, f</i>	<i>compteur 2</i>	cycles
			<i>; si cmpt2 pas 0,</i>	
	<i>goto</i>	<i>boucle2</i>	<i>recommencer boucle1</i>	2 cycles

1) Calculer le temps T_2 que mettra le processeur terminer ce code.

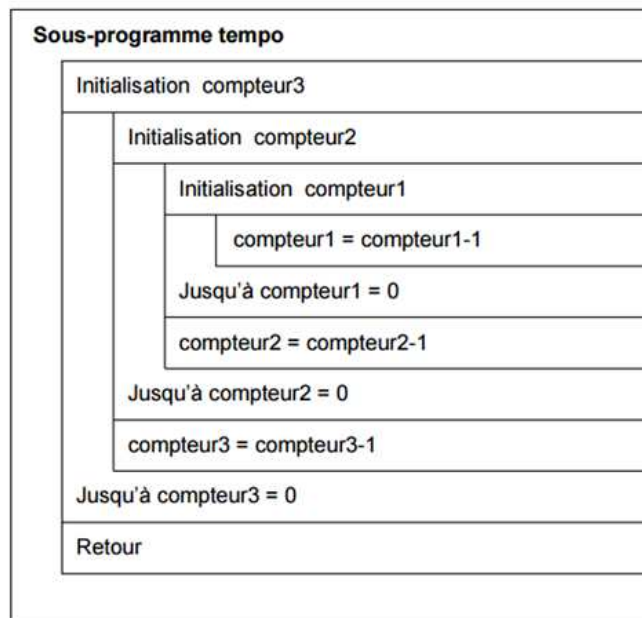
Temporisation avec deux boucles imbriquées

Soit le code suivant :

tempo	movlw	d'10'		1 cycle
	movwf	cmpt3	; initialiser compteur3	1 cycle
boucle3				
	movlw	d'100'		1 cycle
	movwf	cmpt2	; initialiser compteur2	1 cycle
boucle2				
	movlw	d'249'		1 cycle
	movwf	cmpt1	; initialiser compteur1	1 cycle
boucle1				
	nop		; perdre 1 cycle	1 cycle
	decfsz	cmpt1, f	; décrétement compteur1	1 ou 2 cycles
	goto	boucle1	; si pas 0, boucler	2 cycles
	decfsz	cmpt2, f	; si 0, décrétement compteur 2	1 ou 2 cycles
	goto	boucle2	; si cmpt2 pas 0, recommencer boucle1	2 cycles
	decfsz	cmpt3, f	; si 0, décrétement compteur 3	1 ou 2 cycles
	goto	boucle3	; si cmpt3 pas 0, recommencer boucle2	2 cycles

2) Calculer le temps T_3 que mettra le processeur terminer ce code

La structure de la boucle



Annexe

MOVWF	Move W to f			
Syntax:	MOVWF f{,a}			
Operands:	$0 \leq f \leq 255$ $a \in [0,1]$			
Operation:	$(W) \rightarrow f$			
Status Affected:	None			
Encoding:	0110	111a	ffff	ffff
Description:	Move data from W to register 'f'. Location 'f' can be anywhere in the 256-byte bank. If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank (default).			

MOVF	Move f				
Syntax:	MOVF f{,d{,a}}				
Operands:	$0 \leq f \leq 255$ $d \in [0,1]$ $a \in [0,1]$				
Operation:	$f \rightarrow \text{dest}$				
Status Affected:	N, Z				
Encoding:	<table><tr><td>0101</td><td>00da</td><td>ffff</td><td>ffff</td></tr></table>	0101	00da	ffff	ffff
0101	00da	ffff	ffff		
Description:	The contents of register 'f' are moved to a destination dependent upon the status of 'd'. If 'd' is '0', the result is placed in W. If 'd' is '1', the result is placed back in register 'f' (default). Location 'f' can be anywhere in the 256-byte bank. If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank (default).				

CPFSEQ	Compare f with W, Skip if f = W			
Syntax:	CPFSEQ f{,a}			
Operands:	$0 \leq f \leq 255$ $a \in [0,1]$			
Operation:	$(f) - (W)$, skip if $(f) = (W)$ (unsigned comparison)			
Status Affected:	None			
Encoding:	0110	001a	ffff	ffff
Description:	Compares the contents of data memory location 'f' to the contents of W by performing an unsigned subtraction. If 'f' = W, then the fetched instruction is discarded and a NOP is executed instead, making this a two-cycle instruction. If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank (default).			

CPFSLT	Compare f with W, Skip if f < W				
Syntax:	CPFSLT f {,a}				
Operands:	0 ≤ f ≤ 255 a ∈ [0,1]				
Operation:	(f) − (W), skip if (f) < (W) (unsigned comparison)				
Status Affected:	None				
Encoding:	<table><tr><td>0110</td><td>000a</td><td>ffff</td><td>ffff</td></tr></table>	0110	000a	ffff	ffff
0110	000a	ffff	ffff		
Description:	Compares the contents of data memory location 'f' to the contents of W by performing an unsigned subtraction. If the contents of 'f' are less than the contents of W, then the fetched instruction is discarded and a NOP is executed instead, making this a two-cycle instruction. If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank (default).				

CPFSGT	Compare f with W, Skip if f > W				
Syntax:	CPFSGT f{,a}				
Operands:	$0 \leq f \leq 255$ $a \in [0,1]$				
Operation:	$(f) - (W)$, skip if $(f) > (W)$ (unsigned comparison)				
Status Affected:	None				
Encoding:	<table border="1"><tr><td>0110</td><td>010a</td><td>ffff</td><td>ffff</td></tr></table>	0110	010a	ffff	ffff
0110	010a	ffff	ffff		
Description:	Compares the contents of data memory location 'f' to the contents of the W by performing an unsigned subtraction. If the contents of 'f' are greater than the contents of WREG, then the fetched instruction is discarded and a NOP is executed instead, making this a two-cycle instruction. If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank (default).				

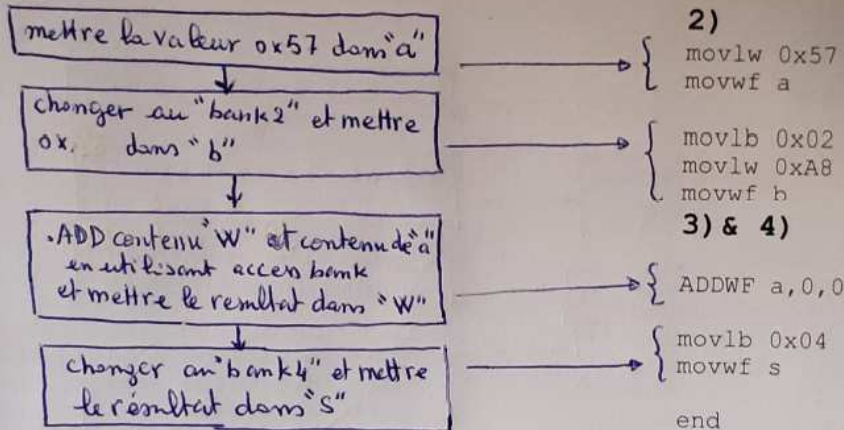
Corrigé TD3

Exercice 1

```
list p=18f4550, r=dec
include <p18f4550.inc>
```

1)

```
a    equ 0x020 ; adresse de var1
b    equ 0x2FF ; adresse de var 2
s    equ 0x400 ; adresse de sum
```



Exercice 3

$$1) T_2 = \underbrace{1+1}_{\text{initialisation du 'cmt2'}} + \underbrace{[T_1 + 1 + 2]}_{T_1 + \text{decfsz}(1)} \times \underbrace{(\text{cmt2} - 1)}_{(\text{cmt2} - 1)} + \underbrace{T_1 + 2}_{T_1 + \text{decfsz}(2)}$$

on rappelle que $T_1 = 4 \cdot \text{cmt1} + 1$ (Ex3TD2) qui est égal ici à 997 cycles

$$\Rightarrow T_2 = (T_1 + 3) \times 99 + T_1 + 4 = (T_1 + 3) \cdot \text{cmt2} + 1 = 100001 \text{ cycles}$$

avec un quartz 4MHz $\rightarrow T_{cy} = 1\mu s \rightarrow T_2 \approx 100ms = 0,1s$

$$2) T_3 = \underbrace{1+1}_{\text{initialisation cmt3}} + \underbrace{(T_2 + 1 + 2)}_{T_2 + \text{decfsz}(2) + \text{goto}(2)} \times (\text{cmt3} - 1) + \underbrace{T_2 + 2}_{T_2 + \text{decfsz}(2)}$$

$$\Rightarrow T_3 = (T_2 + 3) \cdot \text{cmt3} + 1 = 1000041 \text{ cycles} \approx 1 \text{ seconde}$$

Exercice 2

1)

```
list p=18f4550 , r=dec
include <p18f4550.inc>
a      equ 0x020 ; adresse de var1
b      equ 0x2FF ; adresse de var 2
r      equ 0x400 ; adresse de sum
```

2)

